

From Agent Traces to Analytics

Building agent eval pipelines with OSS Python



DAY 1

evaluation_dataset_v1.csv

```
task_id,prompt,expected_tool,expected_output
001,"Load NYC taxi data",dlt.run,success
002,"Sync Postgres -> BigQuery",dlt.run,success
003,"Parse this CSV",dlt.from_file,success
004,"Add incremental loading",dlt.run,success
005,"...",...,...
```

FROZEN

reality
drift

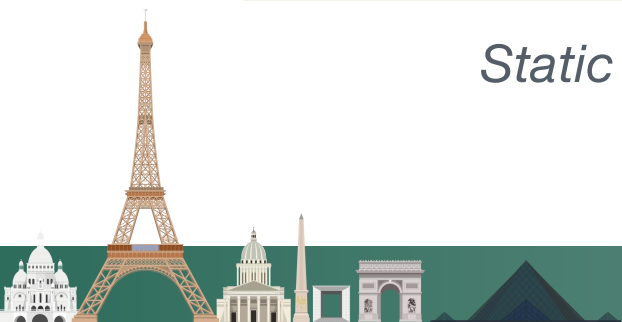
DAY 30 — PRODUCTION

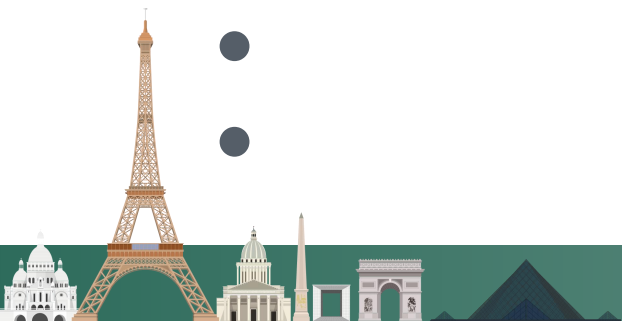
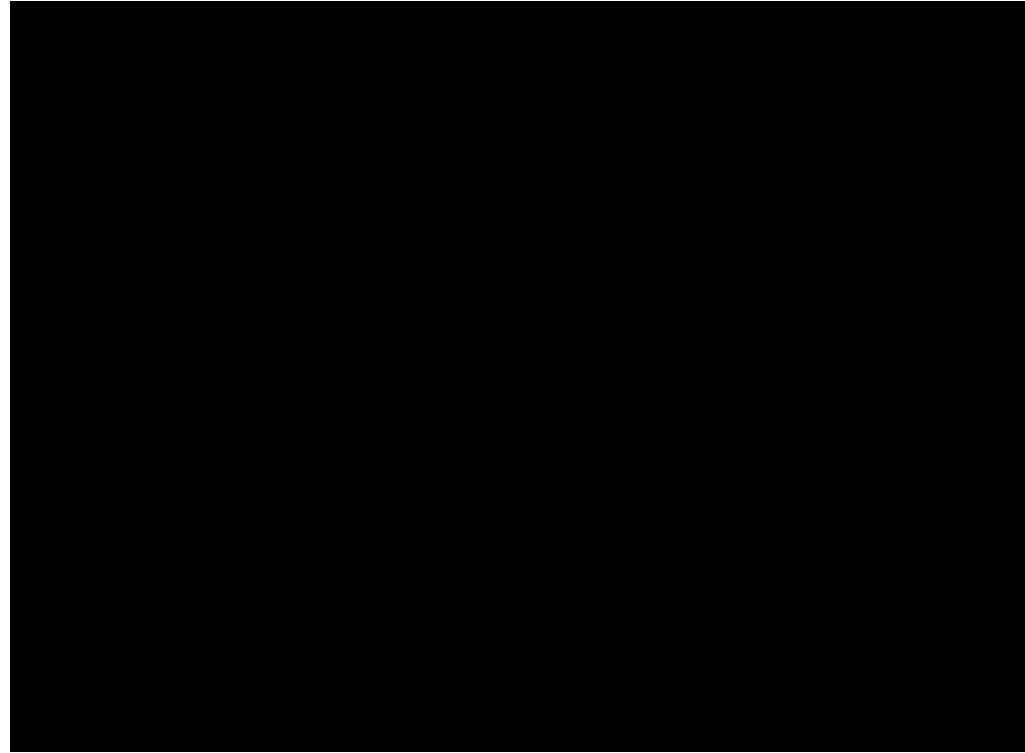
real agent traces

```
user: "can u sync the stripe stuff to snowflake idk"
  → dlt.run() retry × 3, schema drift, OOM
user: "why is my pipeline broken at 3am"
  → tool not in v1 catalog: dlt.transformations
user: "compare last month vs this month"
  → 14-turn loop, $4.20, ½ the prompts new
user: "add ingestion for our internal API"
  → custom auth, paginator, no eval coverage
...
```

● LIVE STREAM

Static datasets can't keep up with how people actually use your agent.





dltHub



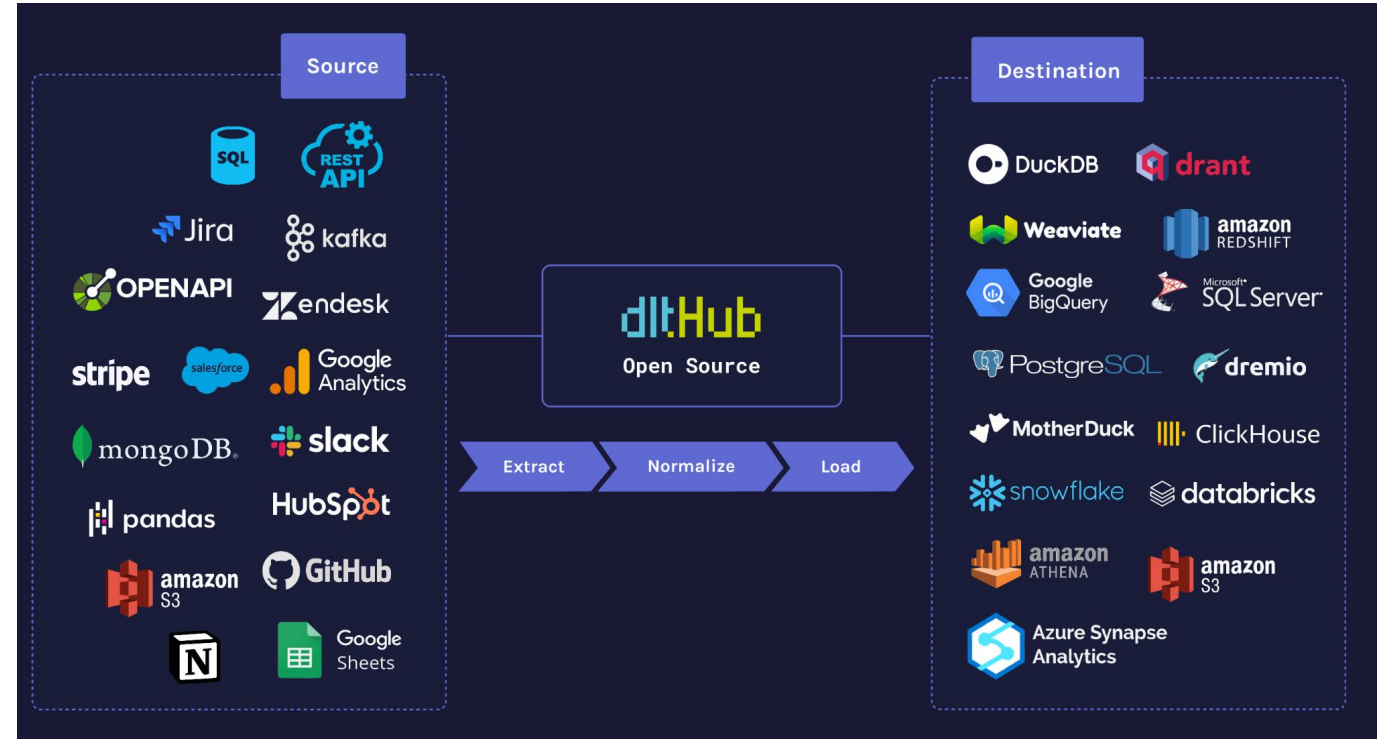
dlt contains all DE best practices. Any engineer now can create the pipeline on a senior level:

- Schema evolution
- Data contracts
- Incremental loading
- Performance management

dlt has particular PMF across two diagonals:

- Managing the long tail of data sources (niche and proprietary REST APIs) & customisation of data sources
- Replicating production databases reliably and efficiently

pip install dlt



dlt is the most popular ingestion Python library

GOSIM Paris 2026

Monthly Custom Sources Created by Users '24 - '26




```
import dlt
from dlt.sources.rest_api import rest_api_resources
from dlt.sources.rest_api.typing import RESTAPIConfig

@dlt.source
def github_source(access_token=dlt.secrets.value):
    config: RESTAPIConfig = {
        "client": {
            "base_url": "https://api.github.com",
            "auth": {"type": "bearer", "token": access_token},
            "headers": {"Accept": "application/vnd.github.v3+json"}
        },
        "resources": ["issues", "pull_requests"],
        "resource_defaults": {
            "primary_key": "id",
            "write_disposition": "merge"
        }
    }
    yield from rest_api_resources(config)

if __name__ == "__main__":
    pipeline = dlt.pipeline("github_ingestion", destination="duckdb")
    pipeline.run(github_source)
```



dltHub AI Agent

PROMPT

Build me a pipeline from the GitHub API into DuckDB, with incremental loading by repo.

AGENT

```
search_dlt_docs('github api source')
read_template('rest_api')
write_pipeline.py (24 lines)
python pipeline.py → ✓ 47 repos loaded
add incremental: cursor=updated_at
python pipeline.py → ✓ 0 new (correct)
```

OUTPUT

✓ pipeline.py





dltHub AI Agent

PROMPT

Build me a pipeline from the GitHub API into DuckDB, with incremental loading by repo.

AGENT

```
search_dlt_docs('github api source')
read_template('rest_api')
write_pipeline.py (24 lines)
python pipeline.py → ✓ 47 repos loaded
add incremental: cursor=updated_at
python pipeline.py → ✓ 0 new (correct)
```

OUTPUT

✓ pipeline.py



Outcome

- Does the pipeline run?
- Does data actually load?
- Schema correctness
- Row-count parity



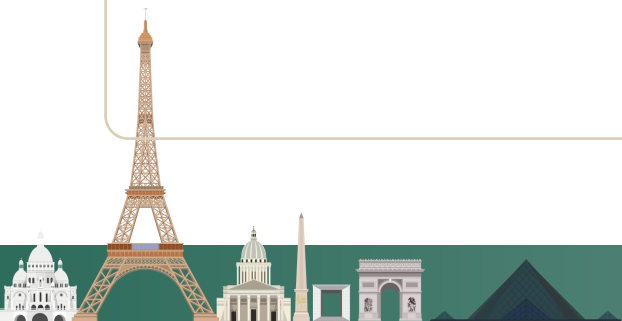
Behavioral

- Tool-call sequences
- Retry patterns
- Tool-transition matrix
- Testing patterns



Economic

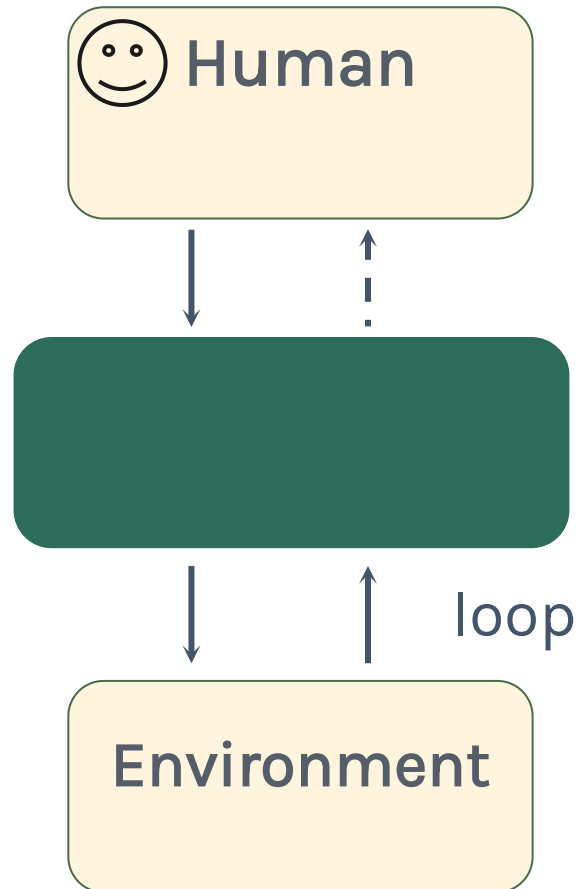
- Tokens in / out
- Turns per task
- Latency (p50, p95)
- \$ per successful run



Set up a **fresh, multimodal dataset** of agent's behavior and use it to **evaluate, debug, and improve**.

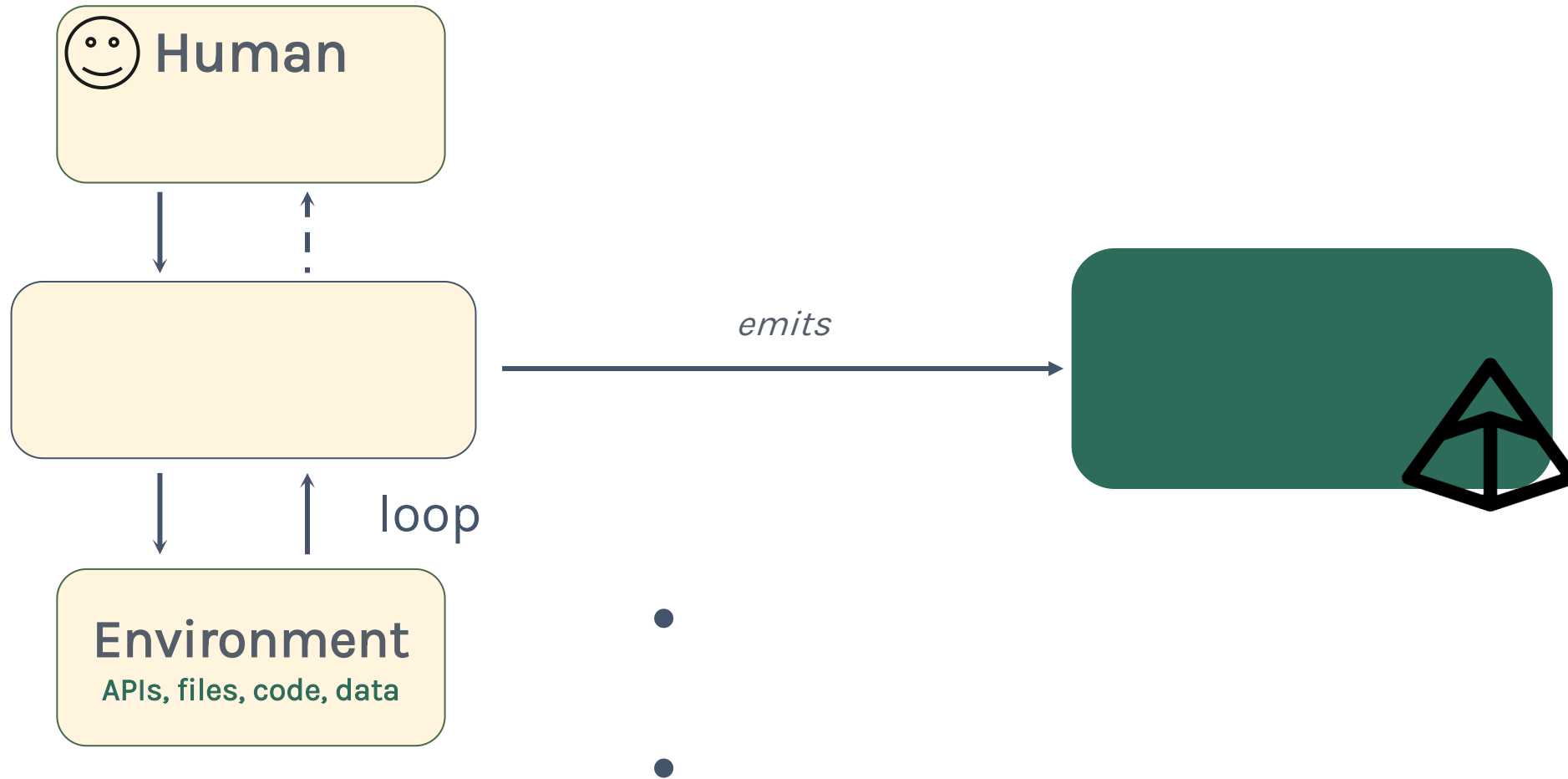


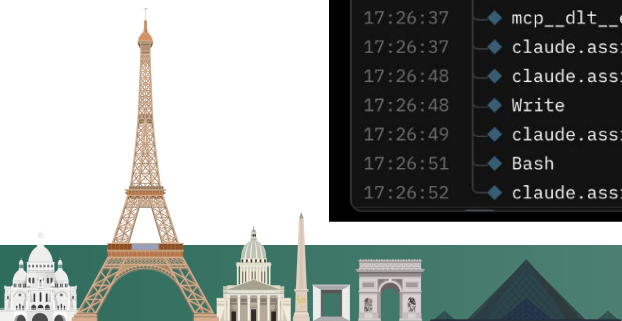
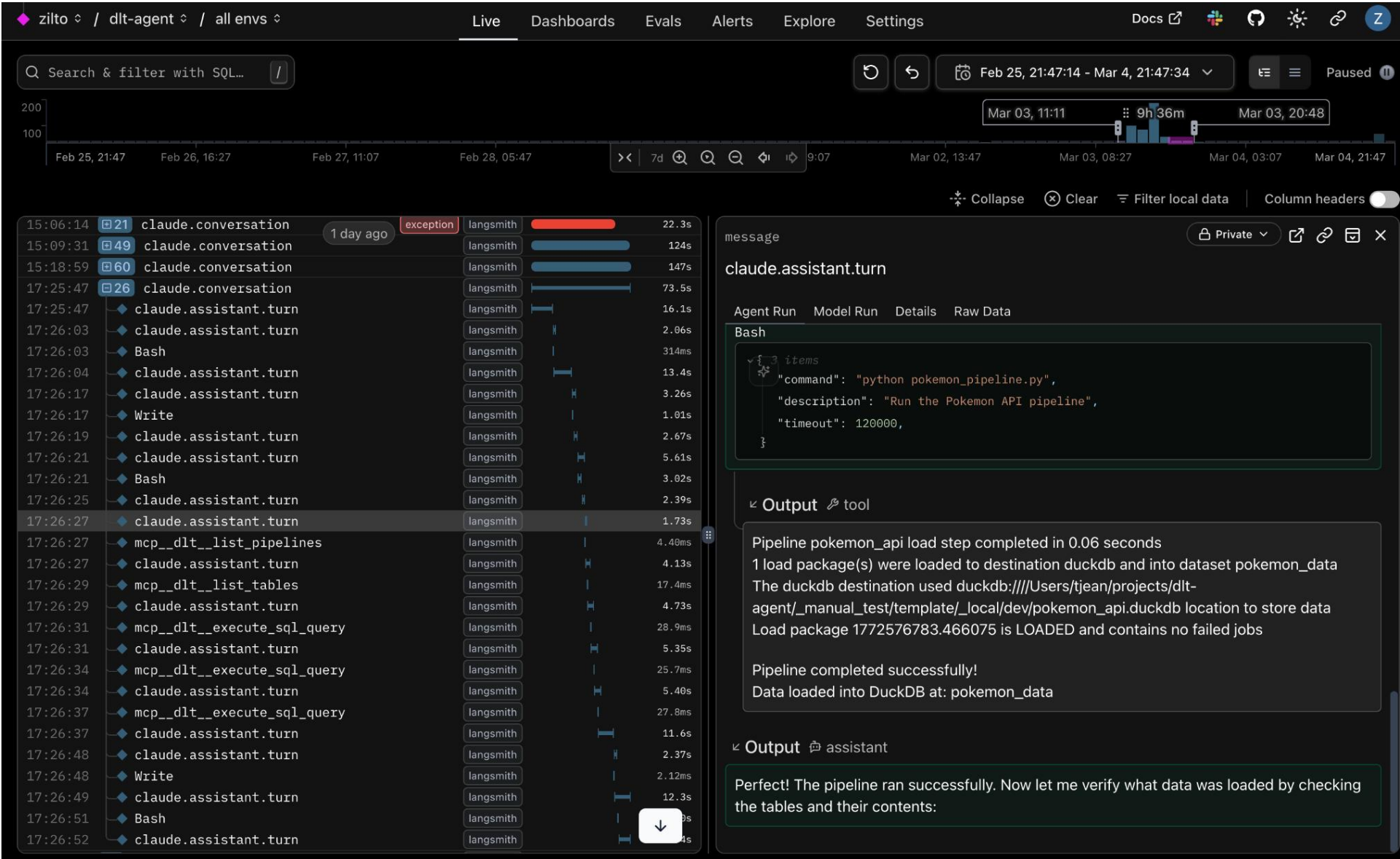
Agents are loops – but can you see inside?



-
-
-
-
-
-

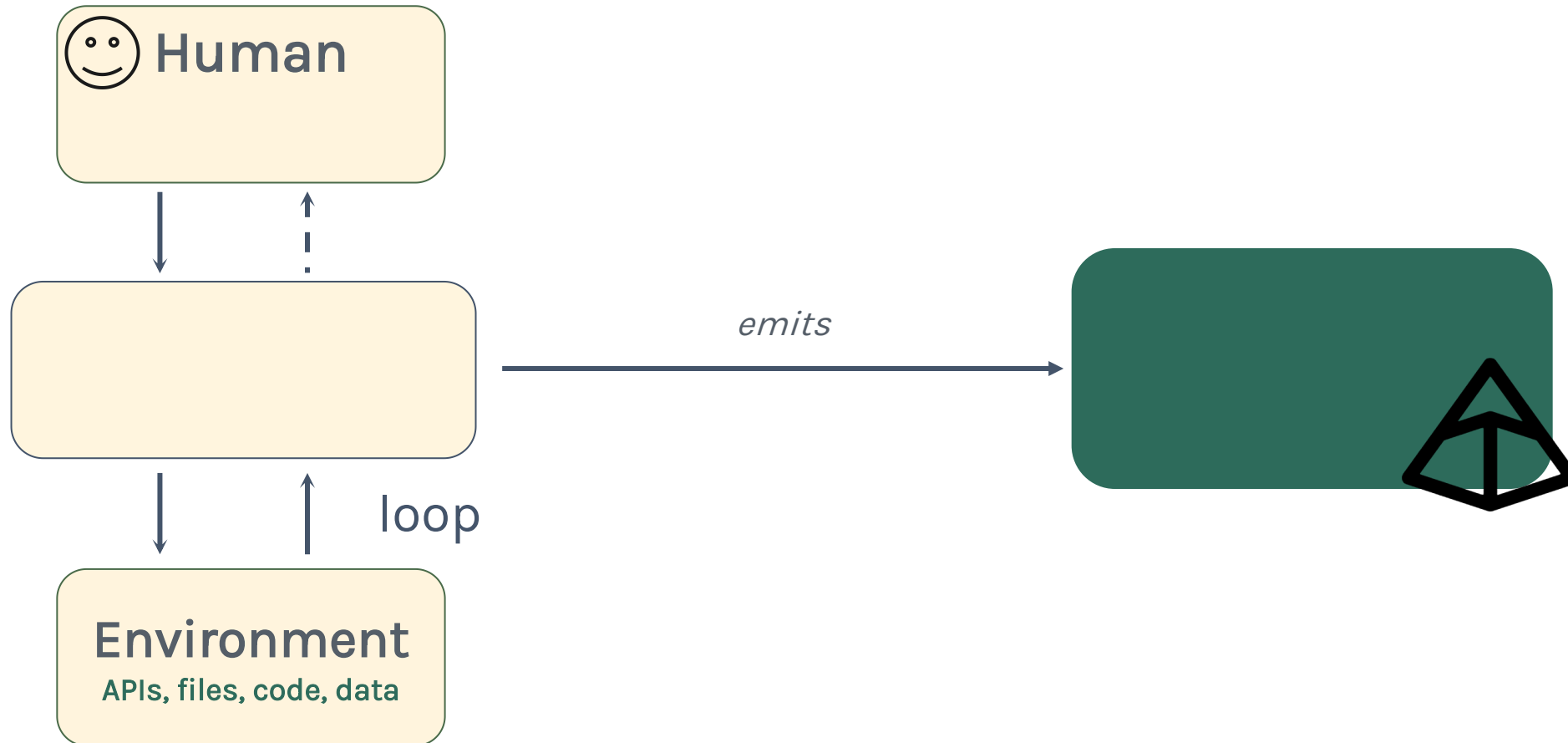




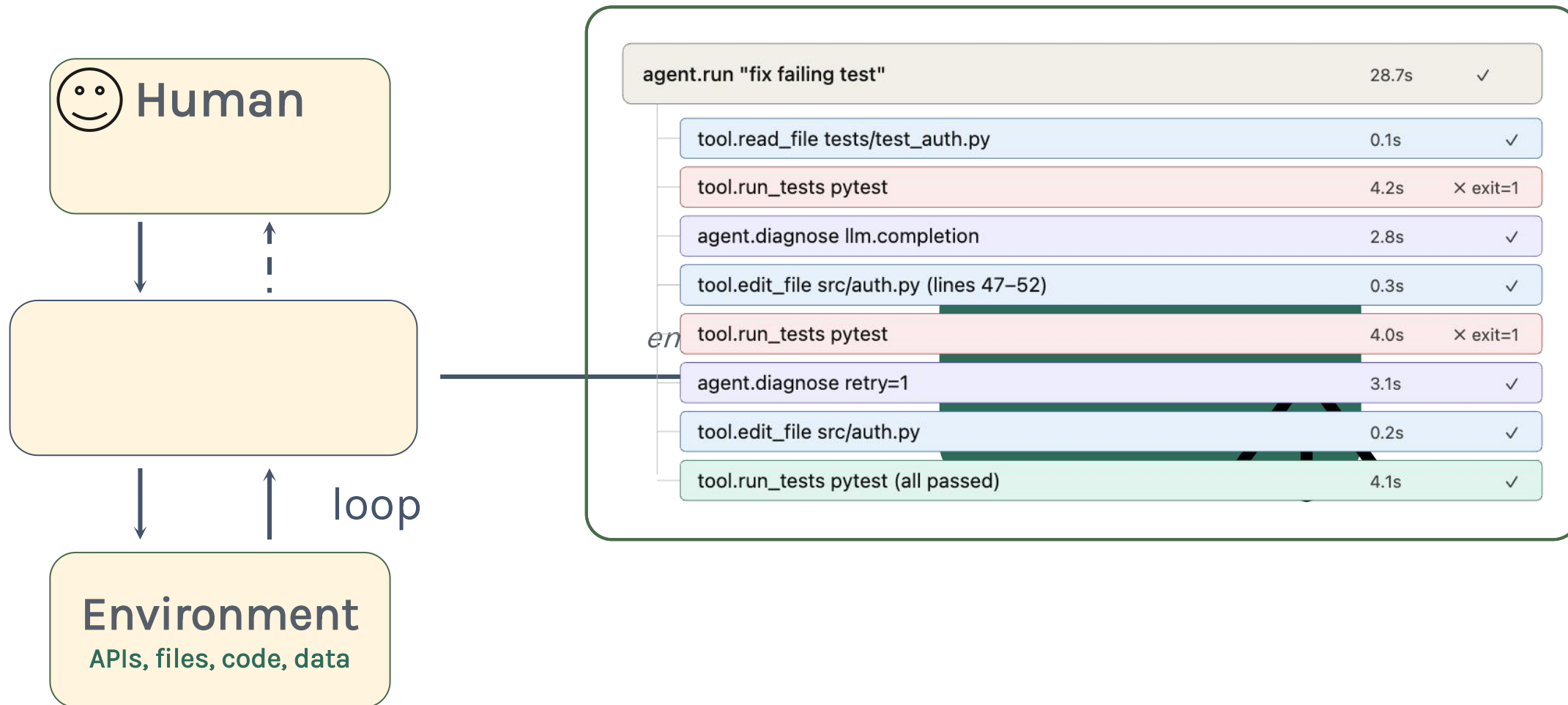


The challenge – logs are nested dictionaries

GOSIM Paris 2026



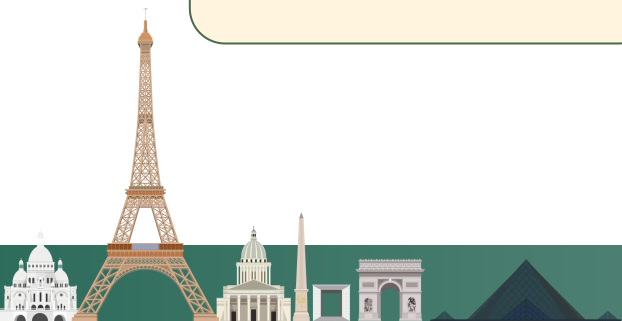
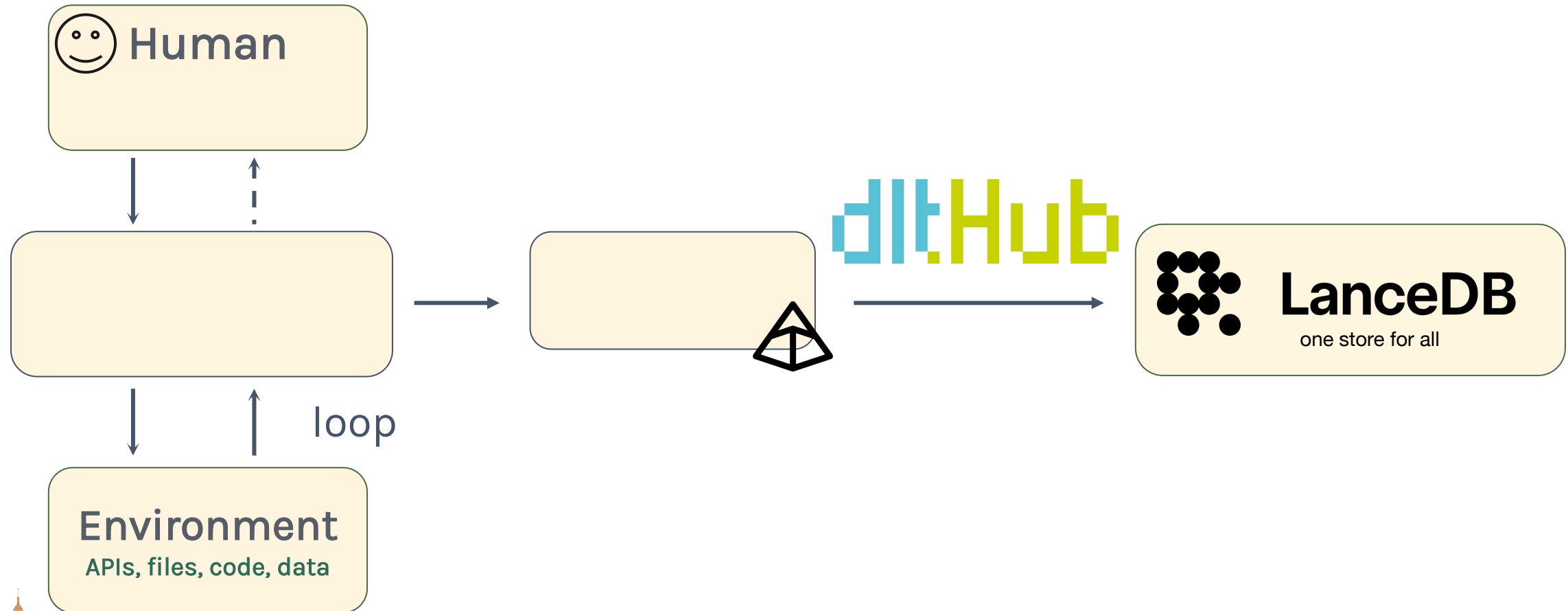
The challenge – logs are nested dictionaries



```
{
  "name": "agent.run", "duration_ms": 14820, "status": "ok",
  "attrs": { "agent": "support-triage", "model": "claude-opus-4-7" },
  "children": [
    { "name": "planner.decompose", "duration_ms": 812,
      "children": [
        { "name": "llm.completion", "duration_ms": 784,
          "attrs": { "tokens.in": 412, "tokens.out": 156 } }
      ] },
    { "name": "subagent.researcher", "duration_ms": 6240,
      "children": [
        { "name": "tool.search_kb", "duration_ms": 312,
          "attrs": { "query": "refund policy", "hits": 7 },
          "children": [
            { "name": "vector.embed", "duration_ms": 94 },
            { "name": "vector.query", "duration_ms": 182,
              "attrs": { "index": "kb-prod", "top_k": 10 } }
          ] },
        { "name": "tool.web_fetch", "duration_ms": 2104, "status": "error",
          "attrs": { "url": "https://...", "error": "timeout" } },
        { "name": "tool.web_fetch", "duration_ms": 1487, "attrs": { "retry": 1 } },
        { "name": "llm.summarize", "duration_ms": 2204,
          "children": [
            { "name": "guardrail.pii_check", "duration_ms": 48,
              "attrs": { "violations": 0 } },
            { "name": "guardrail.toxicity", "duration_ms": 36 }
          ] }
      ] },
    { "name": "agent.reflect", "duration_ms": 1408,
      "attrs": { "tokens.in": 3847, "tokens.out": 284 } },
    { "name": "tool.create_ticket", "duration_ms": 6204,
      "attrs": { "ticket.id": "T-44219", "priority": "P2" },
      "children": [
        { "name": "http.request", "duration_ms": 5980,
          "attrs": { "method": "POST", "status": 201 } }
      ] }
  ]
}
```

est"	28.7s	✓
ts/test_auth.py	0.1s	✓
test	4.2s	✗ exit=1
m.completion	2.8s	✓
auth.py (lines 47-52)	0.3s	✓
test	4.0s	✗ exit=1
etry=1	3.1s	✓
auth.py	0.2s	✓
test (all passed)	4.1s	✓





Traces are only 30% of the story

GOSIM Paris 2026

OTel traces

Logfire spans · attributes

Code the agent wrote

unstructured text

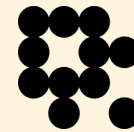
Files it produced

blobs

Operational logs

errors · retries · system state

This is rich data that we want to version and track in a single place



LanceDB
one store for all

Multimodal usually means vision + audio.

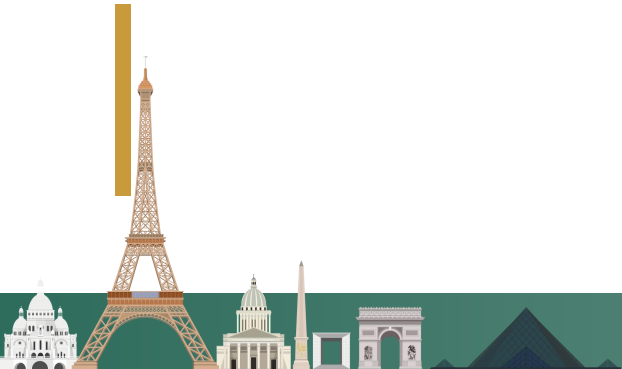
But agent output is also multimodal: **structured traces, unstructured code, file blobs, logs.**

We want everything to be stored in one place and queryable as a table

<https://lancedb.com/blog/what-we-mean-by-multimodal/>



session_id	trace	code	file_blob	embedding	metadata
sess_a91	{spans: 14, ...}	@dlt.resource def gh(): ...	schema.json 2.4 KB	[0.13, -0.42, 0.91, ...]	model=claude-4.7 turns=8 cost=\$0.42
sess_b02	{spans: 22, ...}	import dlt from rest_api import...	sample.csv 47 rows	[0.08, 0.55, -0.21, ...]	model=claude-4.7 turns=14 cost=\$1.08
sess_c4f	{spans: 31, ...}	def stripe_source(): ...	screenshot.png 240 KB	[-0.42, 0.18, 0.71, ...]	model=claude-4.7 turns=22 cost=\$2.31



```
import dlt
from logfire.query_client import AsyncLogfireQueryClient

@dlt.resource
async def logfire_records(
    access_token = dlt.secrets.value,
    min_timestamp = dlt.sources.incremental(
        "created_at", initial_value="2026-01-01T00:00:00Z",
    ),
):
    async with AsyncLogfireQueryClient(read_token=access_token) as client:
        yield client.query_arrow(
            sql="SELECT * FROM records LIMIT 1000",
            min_timestamp=min_timestamp.start_value
        )

if __name__ == "__main__":
    pipeline = dlt.pipeline("logfire_export", destination="lancedb")
    pipeline.run(logfire_records, loader_file_format="parquet")
```

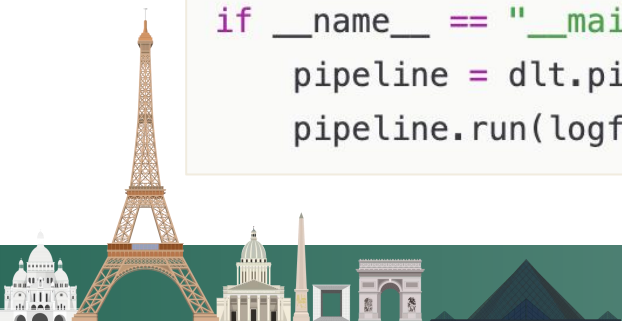
Schema evolution

Incremental loading

One decorator

≈

pip install dlt





Profile: dev

Workspace: pyai-demo

Pipeline: logfire_export

Pipeline logfire_export

Refresh

Open pipeline working directory

Open local data location

Last execution ID: 1754ccba

Total time: 758ms

Executed: 3 days ago

Extract 708ms

Normalize 23ms

Load 25ms

1 dataset migration(s)

succeeded

Status of load packages from last execution

load_id	status
1772834826.7169092	loaded

1 row, 2 columns

<<

<

Page 1 of 1

>

>>

Pipeline Info

Basic properties of the selected pipeline

Dataset: Inspect Schema

Browse the default schema of the selected pipeline

Dataset: Browse Data and State

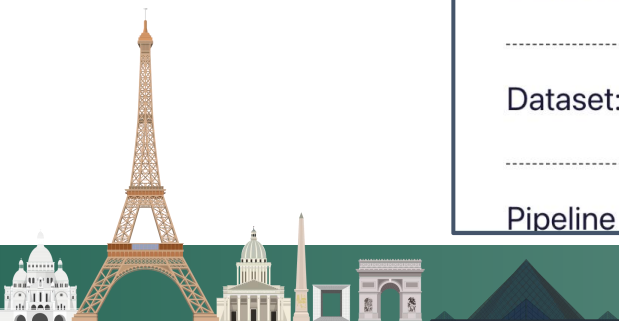
Browse data from the current pipeline.

Dataset: Data Quality

View the results of your data quality checks.

Pipeline State

A raw view of the currently stored pipeline state.



VARIABLES

DATA SOURCES

Search tables...

DuckDB (con)

logfire

logfire_dataset

_dlt_loads

_dlt_pipeline_state

_dlt_version

metrics

metrics_exp_histogram_negative_bucket_c

metrics_exp_histogram_positive_bucket_cc

records 37 columns

created_at timestamp(UTC; 6)

start_timestamp timestamp(UTC; 6)

end_timestamp timestamp(UTC; 6)

duration float64

trace_id string

span_id string

kind string

level int64

parent_span_id string

span_name string

message string

otel_status_code string

otel_links string

otel_events string

/Users/tjean/projects/fde-factory/projects/agent-traces-to-analytics/notebook.py

```
1 import marimo as mo
2 import ibis
3 import dlt
4 import polars as pl
5 import json
```

```
1 pipeline = dlt.pipeline("logfire")
2 dataset = pipeline.dataset()
3 con = dataset.ibis()
```

Review ingested data

1 dataset["records"].df()

Search...

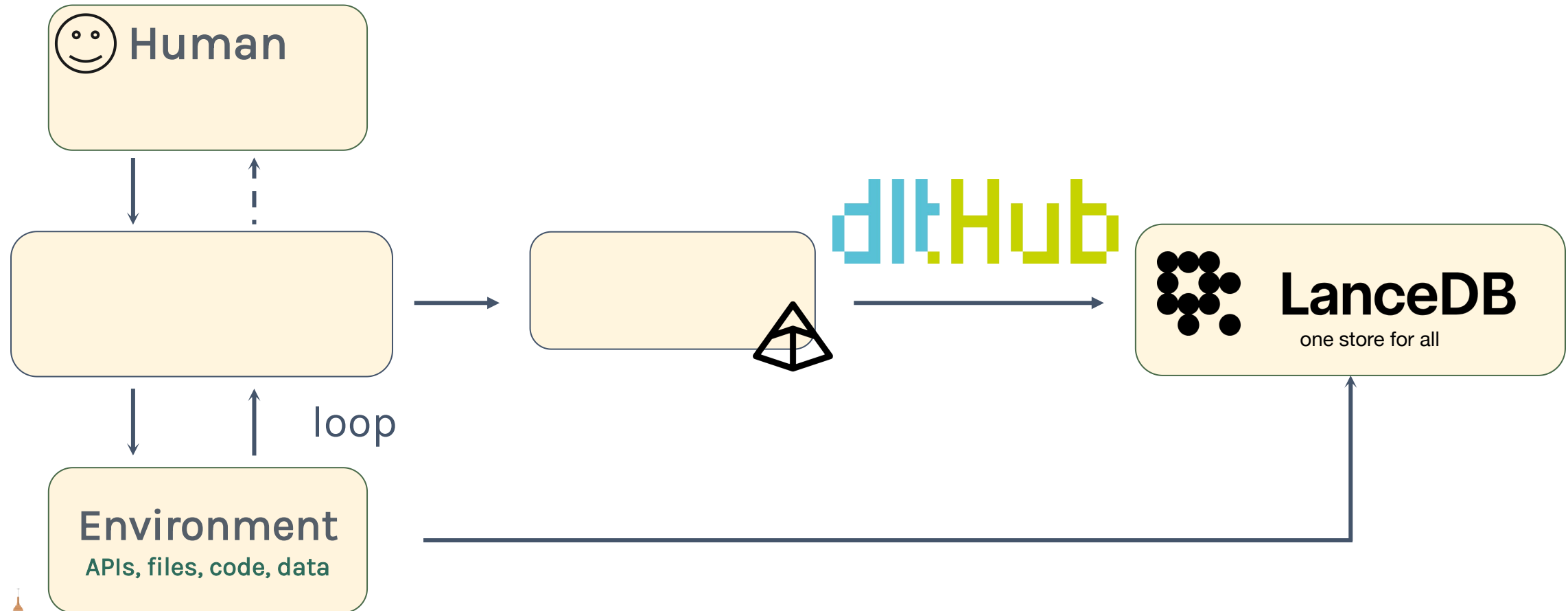
Visualize Explore Export

	trace_id str	span_id str	kind str	level int64	parent_span_id str	span_name str
5.850819	019ddae27d30653de82ff4d9802ae284	35db79eacb77620	span	9	95283007488978f7	chat gpt-5-mini
0.005580	019ddae27d30653de82ff4d9802ae284	396ef559bf12c368	span	17	95283007488978f7	running tool
1.611344	019ddae27d30653de82ff4d9802ae284	6753c6a12939b694	span	9	95283007488978f7	chat gpt-5-mini
0.019490	019ddae27d30653de82ff4d9802ae284	75f0fb0befd425f9	span	17	95283007488978f7	running tool
7.594033	019ddae27d30653de82ff4d9802ae284	942b16083e0a0b84	span	9	95283007488978f7	chat gpt-5-mini
5.093808	019ddae27d30653de82ff4d9802ae284	95283007488978f7	span	9	NaN	agent run
0.002299	019ddae46fa86fe966f5044c38a6105b	045036554266c206	span	17	30f918bad5f72fd9	running tool
1.260868	019ddae46fa86fe966f5044c38a6105b	2bfc195267789608	span	9	30f918bad5f72fd9	chat gpt-5.4-mini-2026-03-17
4.216131	019ddae46fa86fe966f5044c38a6105b	30f918bad5f72fd9	span	9	NaN	agent run
0.022020	019ddae46fa86fe966f5044c38a6105b	87d678bacf6764f1	span	17	30f918bad5f72fd9	running tool

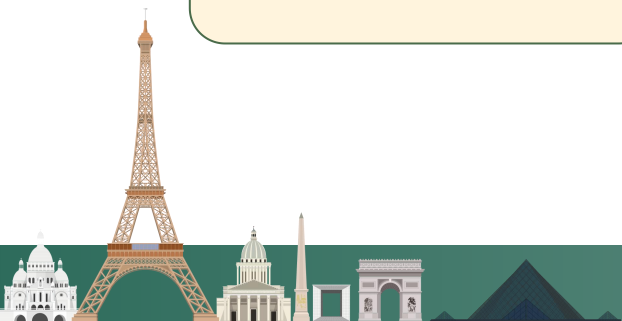
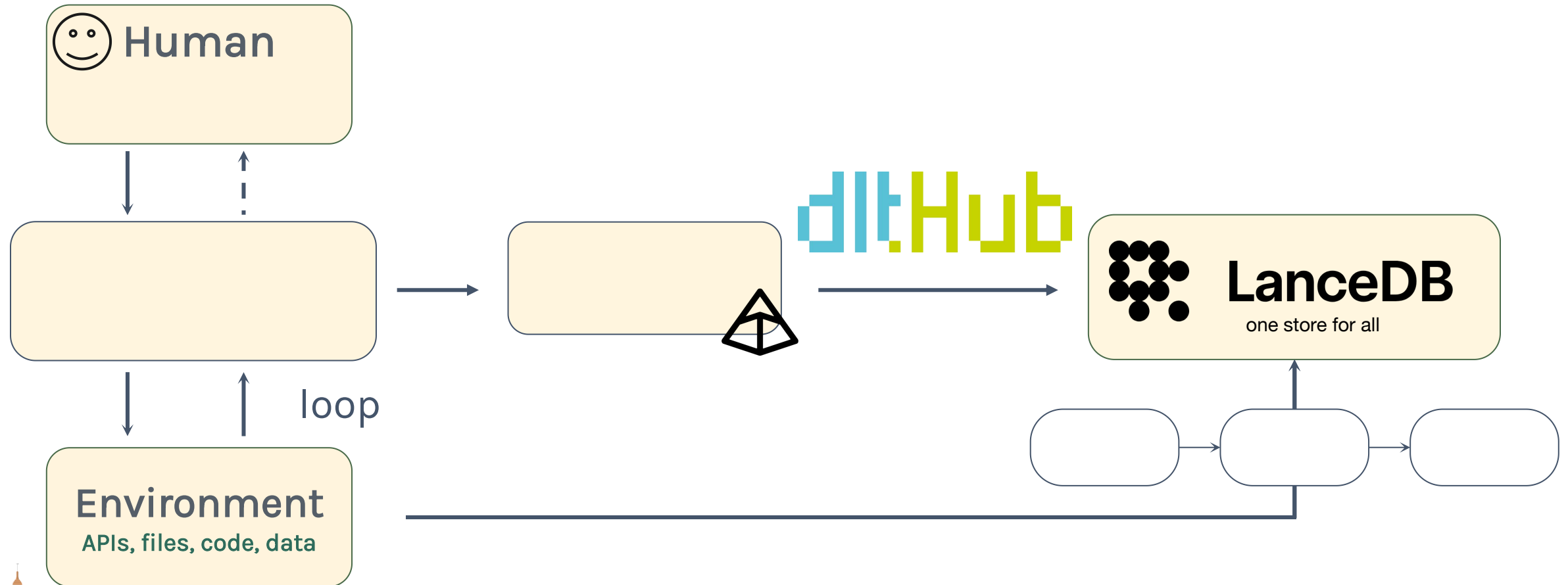
200 rows, 37 columns 10 / page << < Page 1 of 20 > >> No selection

GOSIM

OK, we have data, what about analytics?



OK, we have data, what about analytics?



```
1 @dlt.hub.transformation
2 def span_count_per_run(dataset: dlt.Dataset):
3     query = (
4         dataset.table("records").to_ibis()
5         .agg(span_count=ibis._.count(), by="trace_id")
6     )
7     return query
```

reusable 

```
1 list(span_count_per_run(dataset))[0].df()
```

Search...

 Visualize  Explore  Export

trace_id str unique: 24	span_count  int64
019ddb230a7b7524c74a458f79a1dd5d	15
019ddb1ed3ab9825f2a669675ac15d56	14
019ddb11b73a303c18c54749831a5657	12
019ddafc5a532df95f8acea9f20e1938	11
019ddb0b83b260a01b13cc5f504a8165	11
019ddb153eb2f41e9292e601fa1ce4b9	10
019ddaf135f0dc7f40581d911a032922	10
019ddb1d51dcda05a581fcea8306310d	10
019ddb15e384669600bfae8b3e32a5de	10
019ddb0e7c224b73d5dcf02f30dcb3ea	10

24 rows, 2 columns

10 / page << < Page 1 of 3 > >>

No selection



```
@dlt.hub.transformation
```

```
def tool_transition_matrix(dataset: dlt.Dataset):
```

```
    telemetry = dataset.table("logfile_export").to_ibis()
```

```
    calls = telemetry.filter(telemetry.tool_name.notnull())
```

```
    # rank tool calls within each trace by start_time
```

```
    w = ibis.window(group_by="trace_id", order_by="start_time")
```

Step 1 — number each call

```
    ordered = calls.select(
```

```
        "trace_id", "start_time", from_tool=calls.tool_name, step=ibis.row_number().over(w),
```

```
)
```

```
    # self-join on (trace_id, step+1) to get the next tool
```

Step 2 — pair each call with the next

```
    nxt = ordered.view()
```

```
    pairs = ordered.join(
```

```
        nxt, (ordered.trace_id == nxt.trace_id) & (ordered.step + 1 == nxt.step),
```

```
    ).select(from_tool=ordered.from_tool, to_tool=nxt.from_tool)
```

```
    yield (
```

```
        pairs.group_by(["from_tool", "to_tool"]).aggregate(n_transitions=pairs.from_tool.count())
```

```
        .order_by(ibis.desc("n_transitions"))
```

```
)
```



Add data quality

```
import dlt
from dlt.hub import data_quality as dq

@dq.with_checks(
    dq.checks.is_primary_key("id"),
    dq.checks.is_in(
        "event_type",
        ["system", "user", "agent", "tool"]
    )
)

@dq.with_metrics(
    dq.metrics.column.maximum("turn_number"),
    dq.metrics.column.unique_count("session_id"),
)

@dlt.hub.transformation
def agent_history(dataset: dlt.Dataset):
    ...
```

LAST RUN

1,847 / 1,851

sessions passed contracts

every_session_has_outcome

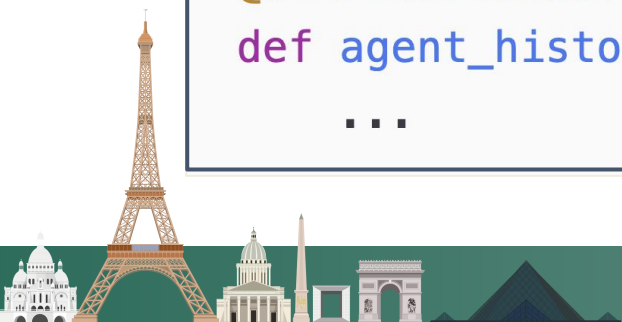


✓ 1851 pass ✓ all pass

final_event_is_terminal

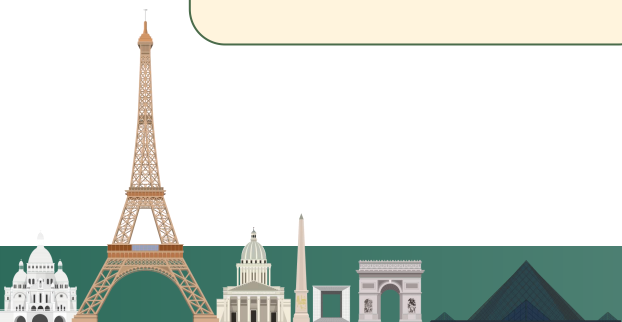
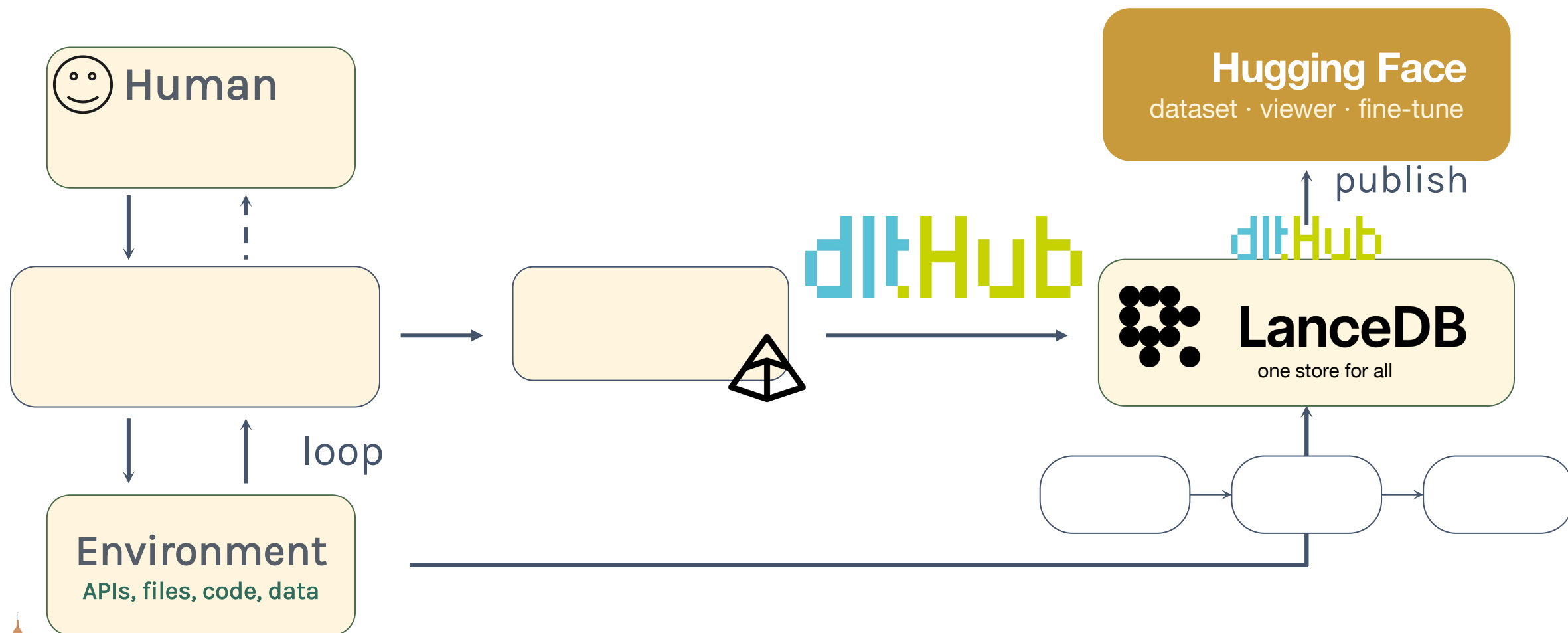


✓ 1847 pass ✗ 4 fail



Once it's clean, the dataset is portable

GOSIM Paris 2026



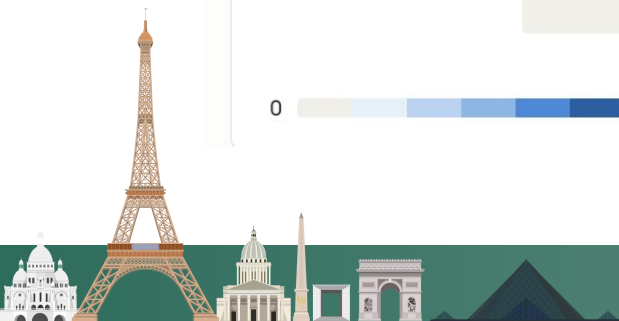


•

•

•

•




```
dlt-runtime v0.26.1
Use "dlt runtime --help-all" for full command reference.

Synced changed workspace files (6 added, 3 updated, 5 deleted)
Synced changed workspace configuration (0 added, 4 updated, 3 deleted)
5 job(s) found in __deployment__
Deployment manifest -- telemetry, metrics, huggingface, agent report

New jobs:
+ telemetry_to_lance (telemetry_to_lance) (trigger: schedule:*/15 * * *
*)
+ lance_to_metrics (lance_to_metrics) (trigger: job.success:jobs.telemetry_to_lance.telemetry_to_lance)
+ lance_to_huggingface (lance_to_huggingface) (trigger: schedule:0 0 * *
0)
+ agent_report (trigger: http:)
```

-
-
-

`dlt runtime deploy`

<https://dlthub.app>





dlthub AI Workbench Jobs

Deployed jobs and their execution history.

Agent eval > dlthub AI Workbench > Jobs

- Overview
- Monitor
- Datasets
- Pipelines
- Explore
- Dashboard
- Notebooks
- MCPs
- Run
- Jobs**
- Deployment & Config

Jobs

All jobs defined in this workspace

Filter by type: All Types

Job	Type		Profile	Trigger	Next Run
dashboard	Interactive		access	HTTP	—
telemetry_to_lance	Batch		prod	Every 15 minutes	in 6 m 5/4/2026, 9:45:00 PM GMT+2
lance_to_metrics	Batch		prod	After telemetry_to_lance	—
lance_to_huggingface	Batch		prod	At 12:00 AM, only on Sunday	in 5 d 5/10/2026, 2:00:00 AM GMT+2
agent_report	Interactive		access	HTTP	—

Per page: 10





Overview

Monitor

Datasets

Pipelines

Explore

Dashboard

Notebooks

MCPs

Run

Jobs

Deployment & Config

jobs.agent_report

Notebook details

Agent eval > dltHub AI Workbench > Notebooks

cell [4]

turns to complete task

turns = dataset.table("turn_count_distribution").df()
mo.ui.altair_chart(bar(turns))

median turns

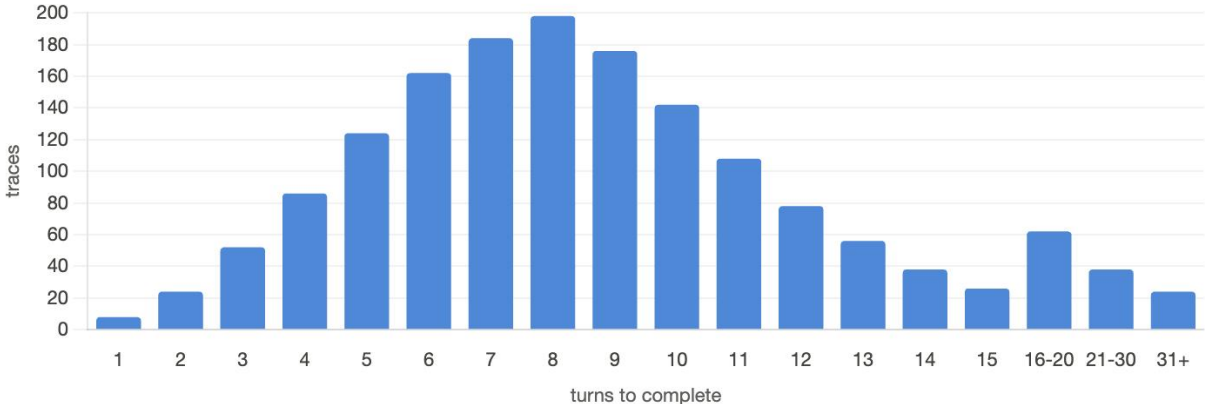
9

p95 turns

26

traces > 30 turns

4.1%



GOSIM

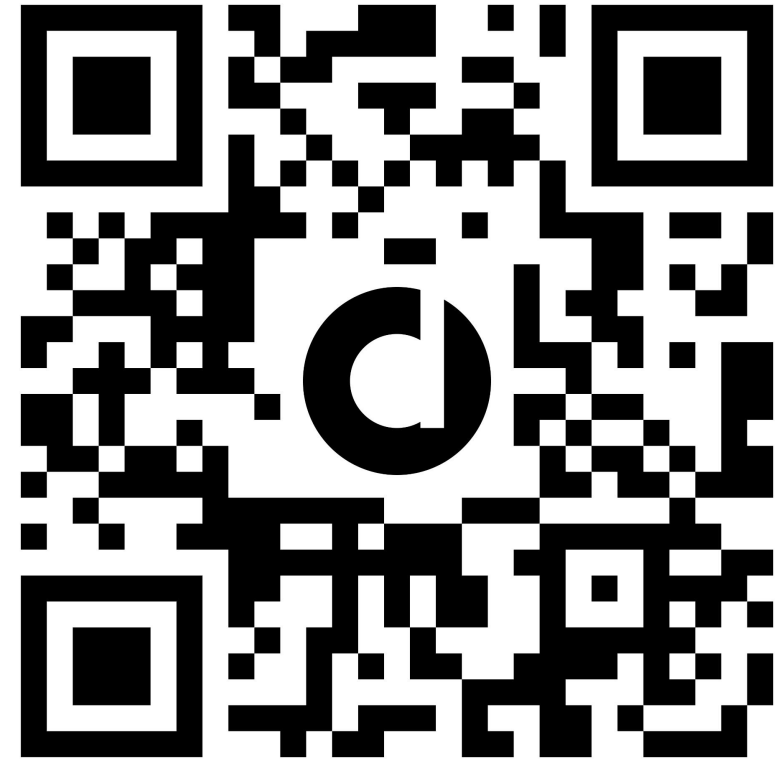
1

2

3

4

`pip install "dlt[hub]"`



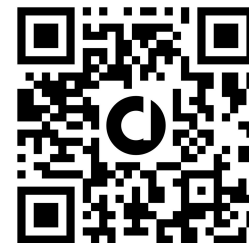
<https://dlthub.com/>



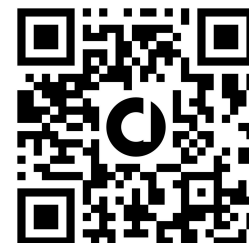
Thanks!

Find me at the workshop track.

Violetta · violetta@dlthub.com



Give us a star



Check out our agent

